Trampolining & Stack Safety

Trampoline · Sum 1..100000 without StackOverflowError · Build with Waqar

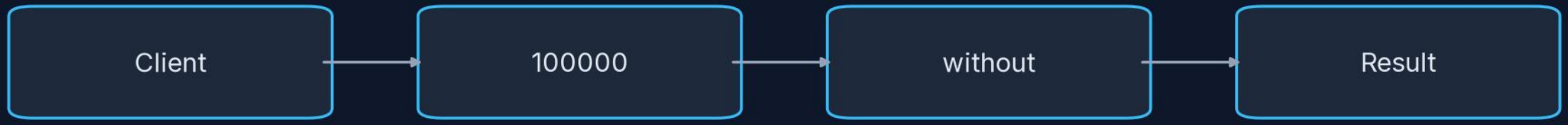
Today's Hook

- Problem: Sum 1..100000 without StackOverflowError
- Pattern: Trampoline — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Sum 1..100000 without StackOverflowError
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Trampoline: Sum 1..100000 without StackOverflowError

Real-World Scenario

- Running example: Sum 1..100000 without StackOverflowError
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.trampoline.problem;

import java.util.*;

public final class NaiveSum {
  public static long sum(long n){ return n<=0?0: n+sum(n-1); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.trampoline.problem;

import java.util.*;
import java.util.function.*;

sealed interface Trampoline<A> permits Done, More {
  A run();
  static <A> Trampoline<A> done(A a){ return new Done<>(a); }
  static <A> Trampoline<A> more(Supplier<Trampoline<A>> next){ return new More<>(next); }
}
record Done<A>(A value) implements Trampoline<A> { public A run(){ return value; } }
record More<A>(Supplier<Trampoline<A>> next) implements Trampoline<A> {
  public A run(){ Trampoline<A> cur=this; while(cur instanceof More<A> m) cur=m.next.get(); return ((Done<A>)cur).value(); }
}
final class SafeSum {
  static Trampoline<Long> sum(long n, long acc){
    return n<=0? Trampoline.done(acc): Trampoline.more(() -> sum(n-1, acc+n));
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.trampoline.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Trampoline: deep sum without SOE ===");
    System.out.println("--- naive (may SOE for large n) ---");
System.out.println(NaiveSum.sum(100));
System.out.println("--- trampoline ---");
System.out.println(SafeSum.sum(100000, 0).run());
  }
}
```

Walkthrough Output

```
=== Trampoline: deep sum without SOE ===
--- naive (may SOE for large n) ---
5050
--- trampoline ---
5000050000
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Trampoline.

Related Patterns

- Contrast Trampoline with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Trampoline → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.